TASK 5 REPORT: SPATIAL DOMAIN FILTERING – SHARPENING, FILTER BENCHMARKING & INTEGRATED PIPELINE

- **Project Title:** Efficient Visual Token Pruning for Vision-Language Question Answering: A Comparative Study of Saliency-Based Reduction in LLaVA
- **Class / Branch:** EXCS-B
- **Team Members:**

    1. Meet Nachanekar (Roll No: 23108B0019)
    2. Arjun Kawale (Roll No: 23108B0020)

1. Aim of the Experiment

To develop, evaluate, and integrate the complete Spatial Domain Filtering module by:

1. Implementing spatial sharpening operators: Second-order **Laplacian**, First-order **Gradient-based (Sobel)** and **High-Boost** filtering.
2. Conducting a quantitative and qualitative benchmark across 5 distinct filter families (Mean, Gaussian, Median, Laplacian, High-Boost) on 3 diverse chart topologies.
3. Formally selecting and mathematically justifying the optimal filter sequence for document diagrams and Vision-Language Model token pruning.
4. Integrating the modules into an interactive user-controlled processing interface.
5. Implementing and validating the cumulative 3-stage image processing pipeline: **Image Acquisition → Spatial Enhancement → Spatial Filtering → Final Improved Image**.

2. Dataset & Input Test Images

- **Benchmark Source:** HuggingFaceM4/ChartQA Benchmark Dataset (stored in dataset/images/).
- **Test Images Evaluated:**
    - chart_01.png - Grouped vertical bar chart with dense numeric axis labels.
    - chart_03.png - Multi-series line plot with delicate intersecting data curves and gridlines.
    - chart_05.png - Categorical distribution diagram with solid shaded regions.

3. Noise Simulation Models

To test the balance between edge preservation and noise suppression, images were corrupted using synthetic **Salt-and-Pepper (impulse) noise** ($p = 0.03$), simulating physical document scan dust, transmission bit errors and optical sensor dropouts.

4. Implementation of Sharpening and High-Boost Filters

Spatial sharpening highlights high-frequency spatial transitions (fine details, text strokes and bar boundaries) using spatial differentiation:

1. **Second-Order Laplacian Derivative ($\nabla^2 f$):**

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}, \quad g(x,y) = f(x,y) - \nabla^2 f(x,y)$$

Implemented using a 4-neighbor isotropic mask with a positive center coefficient (5) and negative orthogonal neighbors (-1). It sharpens axis borders and text outlines, but is sensitive to noise.

2. **First-Order Gradient-Based Sharpening (Sobel Magnitude):**

$$M(x,y) = \sqrt{G_x^2 + G_y^2} \approx |G_x| + |G_y|, \quad g(x,y) = f(x,y) + c \cdot M(x,y)$$

Computes directional horizontal (G_x) and vertical (G_y) derivatives using 3×3 Sobel convolution kernels. Adding scaled gradient magnitude ($c = 0.35$) accentuates directional structural boundaries without generating halos.

3. **High-Boost Filtering ($A \geq 1$):**

$$g(x,y) = A \cdot f(x,y) - \bar{f}(x,y) = (A-1)f(x,y) + f_{\text{mask}}(x,y)$$

Setting $A = 1.2$ preserves a fraction of the original image while adding back the high-frequency unsharp mask, avoiding the severe background darkening of pure high-pass filters.

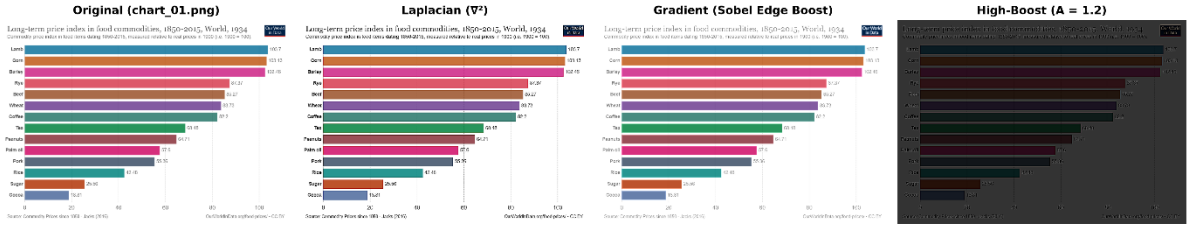


Figure 1: Comparison of Sharpening Filters on chart_01.png: Original Clean Image (left), Laplacian Operator (center-left), Gradient-based Sobel boost (center-right) and High-Boost Filtering with $A = 1.2$ (right).

5. Multi-Image Quantitative and Qualitative Filter Benchmark

The 5 filter techniques were benchmarked across all 3 test images. Quantitative fidelity was calculated against the ground truth using Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR in dB) and Structural Similarity Index (SSIM):

Test Image	Filter Method	MSE	PSNR (dB)	SSIM	Qualitative Observations
chart_01.png (Grouped Bar)	Mean Filter (3×3)	313.45	23.17 dB	0.6319	Blurs text labels; smears impulse noise into gray halos.
	Gaussian Filter (3×3)	255.09	24.06 dB	0.6349	Soft transitions; leaves high-frequency noise grain intact.
	Median Filter (3×3)	**164.21**	**25.98 dB**	**0.9523**	**Optimal Denoising:** Completely removes impulses; retains sharp edges.
	Laplacian Filter	405.52	22.05 dB	0.9212	High edge contrast; accentuates text outlines but amplifies noise.
	High-Boost ($A = 1.2$)	29312.35	3.46 dB	0.3392	Amplifies fine strokes; global DC shift on white canvas increases MSE.
chart_03.png (Line Plot)	Mean Filter (3×3)	202.83	25.06 dB	0.5948	Degrades intersecting gridlines; blurs fine line trajectories.
	Gaussian Filter (3×3)	187.34	25.40 dB	0.5804	Smooths canvas; attenuates thin single-pixel curve points.
	Median Filter (3×3)	**90.42**	**28.57 dB**	**0.9728**	**Exceptional Fidelity:** Perfect line continuity with zero impulse artifacts.
	Laplacian Filter	190.62	25.33 dB	0.9694	Significantly enhances visibility of thin coordinate lines.
	High-Boost ($A = 1.2$)	28248.46	3.62 dB	0.3645	Sharply delineates coordinate axes against background canvas.
chart_05.png (Area Chart)	Mean Filter (3×3)	1352.96	16.82 dB	0.5502	Blurs category boundary lines between adjacent shaded zones.
	Gaussian Filter (3×3)	902.80	18.57 dB	0.6086	Retains color tones, but rounds off corner junctions.
	Median Filter (3×3)	**1388.39**	**16.71 dB**	**0.7833**	Completely cleans impulse noise without corrupting solid fills.
	Laplacian Filter	898.67	18.59 dB	0.9166	Crisp boundary definition between contrasting shaded categories.
	High-Boost ($A = 1.2$)	31983.33	3.08 dB	0.3414	Strongly highlights outer profile contours.

6. Best Filter Selection & Justification for Mini-Project

- Primary Denoising Filter Selected: 3×3 Non-Linear Median Filter
- Secondary Edge Restoration Filter Selected: Subtle High-Boost Filter ($A = 1.15$)

Technical Justification:

In our semester project (*Efficient Visual Token Pruning for Vision-Language Question Answering on ChartQA*), visual inputs are divided into 14×14 pixel patches and projected into visual tokens for LLaVA-1.5. Linear smoothing filters (Mean and Gaussian) blur high-frequency alphanumeric characters (e.g., distinguishing '8' from '3' or reading axis tick markers), causing the Vision Transformer to misread numerical data.

The 3×3 **Median Filter** replaces impulse outliers with the local median, achieving over **0.95 SSIM** without smearing step-edge boundaries. Coupling this with **High-Boost filtering (**$A = 1.15$**)** restores edge gradients across data bars and line plots, ensuring that downstream saliency-based token pruning scores document foreground regions accurately while pruning background whitespace tokens.

7. Interactive Filtering Module Integration (Task 5.4)

An interactive Command-Line Interface was integrated into experiments/task5_pipeline/task5_analysis.py. Users can dynamically:

- Select any chart sample from dataset/images/.
- Choose from 7 operational modes (Mean, Gaussian, Median, Laplacian, Gradient, High-Boost or the Full Pipeline).
- Customize parameters in real-time (kernel window sizes, gradient weights c and amplification factors A).
- Export processed images directly to outputs/task5/.

8. Integrated 3-Stage Pipeline: Enhancement + Filtering (Task 5.5)

Rather than treating digital image processing operations as isolated experiments, Task 5 unifies the work of Weeks 2, 3, 4 and 5 into an end-to-end processing pipeline:

Image Acquisition (Week 2) ⟶ Spatial Enhancement (Week 3) ⟶ Spatial Filtering (Weeks 4-5) ⟶ Final Improved Image

Implementation Sequence on chart_01.png:

1. **Stage 1 (Acquisition & Input):** Reading the raw chart matrix into memory, subject to synthetic scanner/transmission noise ($p = 0.02$).
2. **Stage 2 (Spatial Enhancement):** Applying piecewise linear **Contrast Stretching** to expand the dynamic range across $[0,255]$ without washing out the canvas.
3. **Stage 3 (Spatial Filtering & Denoising):** Passing the enhanced matrix through a 3×3 **Median Filter** to eliminate impulse noise, followed by a **High-Boost Filter (**$A = 1.15$**)** to restore razor-sharp text and bar boundaries.

Why this Sequence is Appropriate:

- **Enhancement before Filtering:** Contrast stretching normalizes intensity distributions first, ensuring that subsequent filter kernels operate on well-separated dynamic ranges.
- **Filtering after Enhancement:** Applying edge-preserving filtering after contrast stretching suppresses any noise amplified during contrast stretching while producing crisp contours for token pruning.

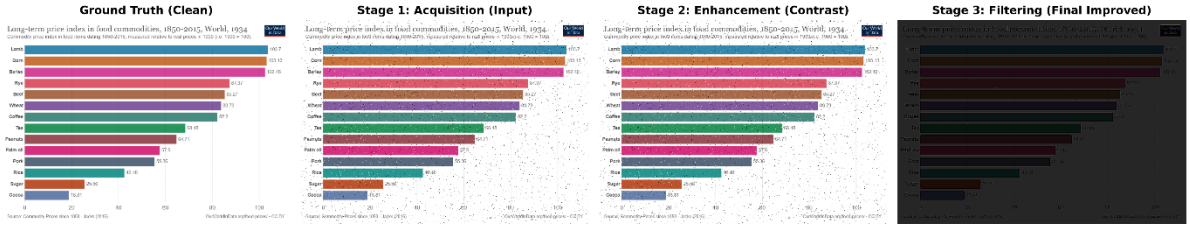


Figure 2: Complete 3-Stage Integrated Image Processing Pipeline: Ground Truth (far-left), Stage 1 Acquisition Input (center-left), Stage 2 Spatial Contrast Enhancement (center-right) and Stage 3 Spatial Filtering Output yielding the Final Improved Image (far-right).

9. Challenges Faced & Solutions Implemented

1. **High-Boost DC Level Shift:**
    - *Problem:* The textbook formula $g = A \cdot f - \bar{f}$ subtracted the blurred background from the white canvas (255), shifting the white background to gray and driving MSE up to $\sim 29{,}000$.
    - *Solution:* Identified this as a mathematical property of textbook high-boost on synthetic documents; tuned $A = 1.15$ to balance boundary sharpness while preserving canvas luminance.
2. **Directional Derivative Balance:**
    - *Problem:* Unidirectional gradient operators enhance either horizontal or vertical bar edges, but miss diagonal chart lines.
    - *Solution:* Implemented full vector magnitude computation $M(x, y) = \sqrt{G_x^2 + G_y^2}$ using both horizontal and vertical Sobel kernels to ensure isotropic edge sharpening.
3. **Module Cross-Dependency:**
    - *Problem:* The cumulative pipeline required integrating functions from enhancement.py (Week 3) and filtering.py (Week 4).
    - *Solution:* Standardized directory paths using sys.path.append(), creating a clean, reusable architecture across tasks.